

OpenACC

A Directive-based Parallel Programming Model

PRESENTED BY

Akash Bansode
CDAC Pune
akashb@cdac.in

Introduction to GPU Computing



OpenACC Programming Model



OpenACC vs CUDA vs HIP



OpenACC Directives



Data Management & Execution Model



Loop Optimization



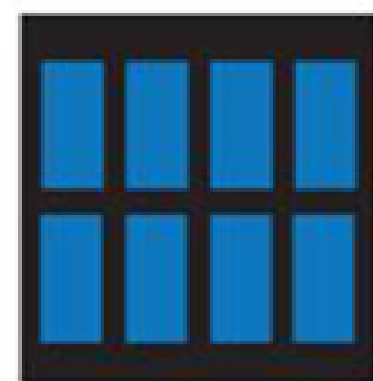
NVIDIA's HPC Compilers



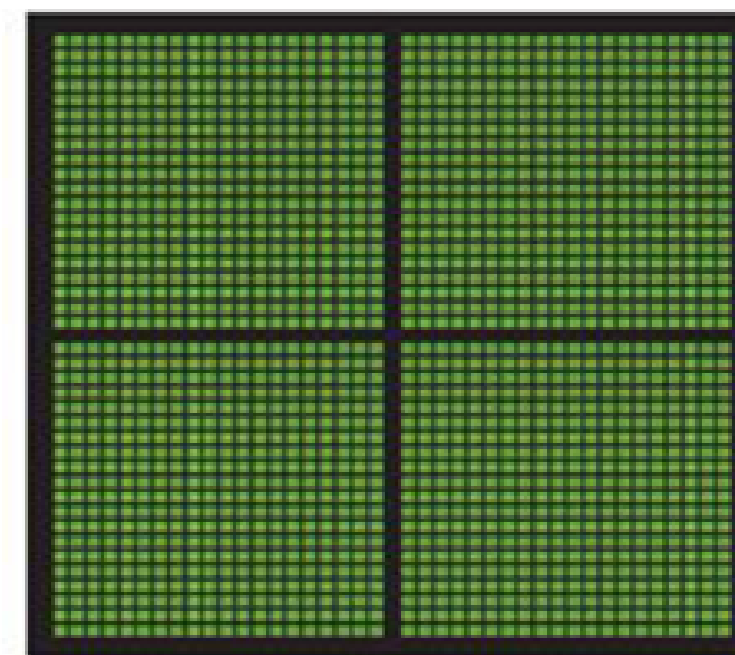
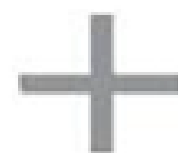
Hands-on Programs



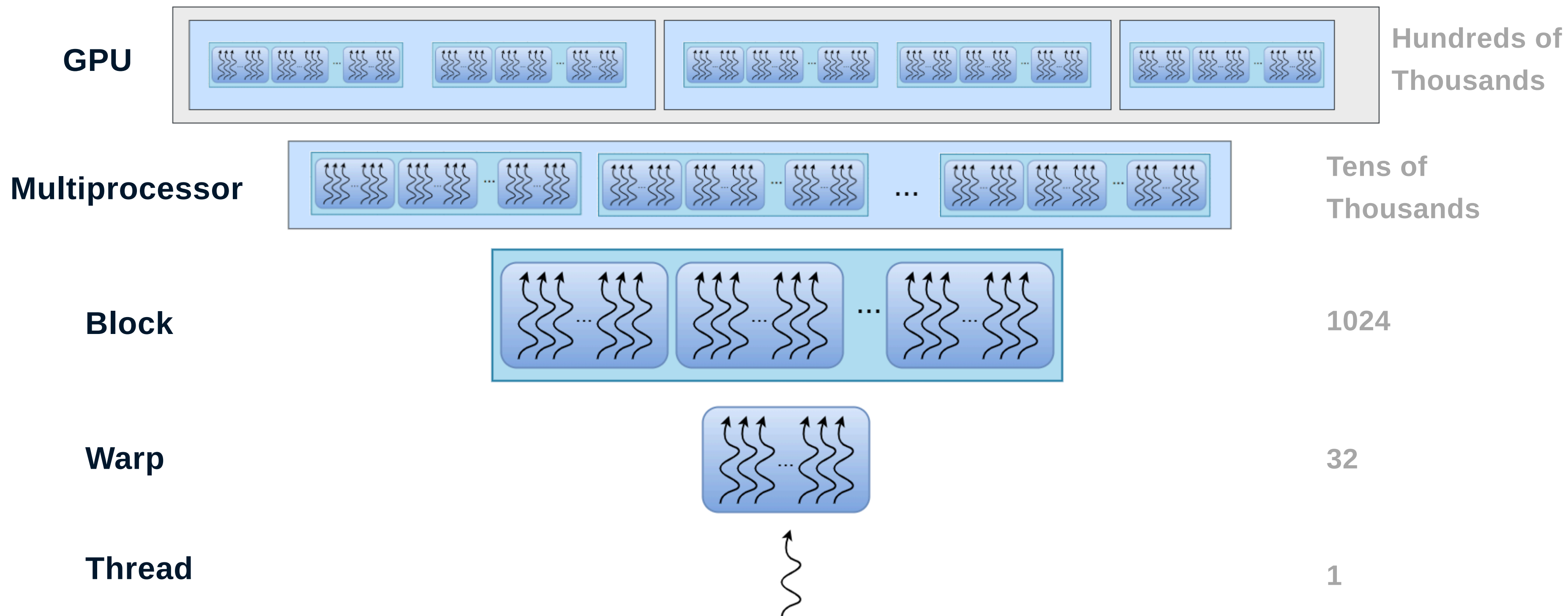
- Originally, **Graphics Processing Unit (GPU)** is dedicated for manipulating computer graphics and image processing.
- Extending the use of the graphics processor to non-graphic workloads known as General Purpose GPU (GPGPU) computing
- CPU and GPU has separated memories. Data transfer between CPU and GPU is required for programming.



CPU
MULTIPLE CORES

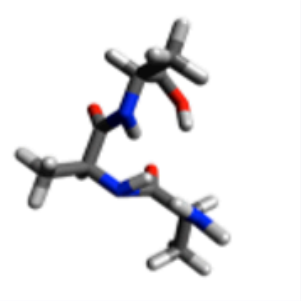


GPU
THOUSANDS OF CORES



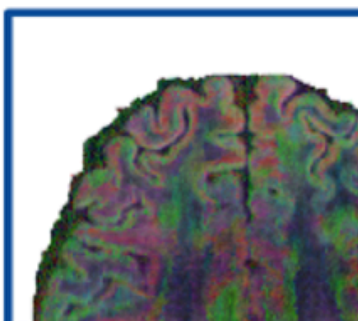
- **OpenACC (Open Accelerators)** is a high-level, directive-based parallel programming model designed that provides portable programming approach for accelerators.
- As such, one can develop a single source code that can be run on a range of devices and achieve good performance.
- OpenACC emerged in 2011 as a programming model that uses high-level compiler directives to expose parallelism in the code and parallelizing compilers to build the code for a variety of parallel accelerators.
- OpenACC defines a set of compiler directives that allow code regions to be offloaded from a host CPU to be computed on a GPU
- Large similarity to OpenMP directives & support for both C/C++ and Fortran bindings.

OpenACC
Directives for Accelerators



LSDalton

Quantum Chemistry
Aarhus University
12X speedup
1 week



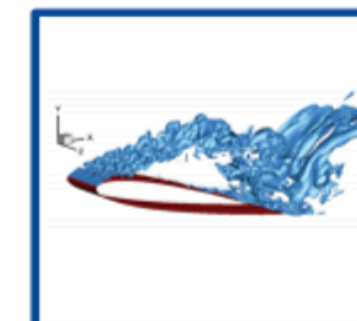
PowerGrid

Medical Imaging
University of Illinois
40 days to
2 hours



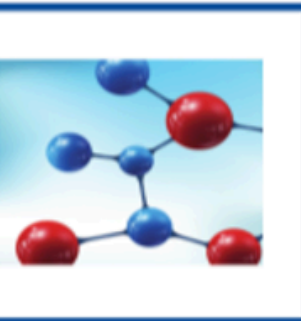
COSMO

Weather and Climate
MeteoSwiss, CSCS
4X speedup
3X energy efficiency



INCOMP3D

CFD
NC State University
4X speedup



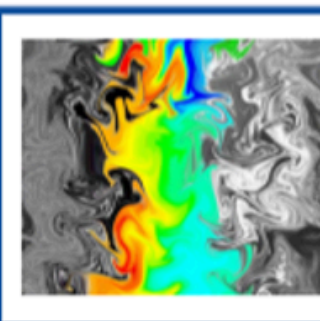
NekCEM

Comp Electromagnetics
Argonne National Lab
2.5X speedup
60% less energy



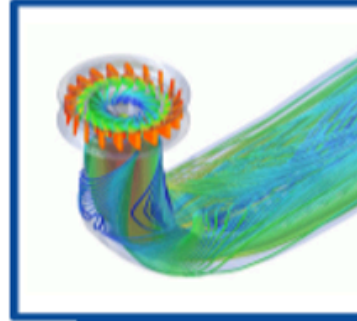
MAESTRO CASTRO

Astrophysics
Stony Brook University
4.4X speedup
4 weeks effort



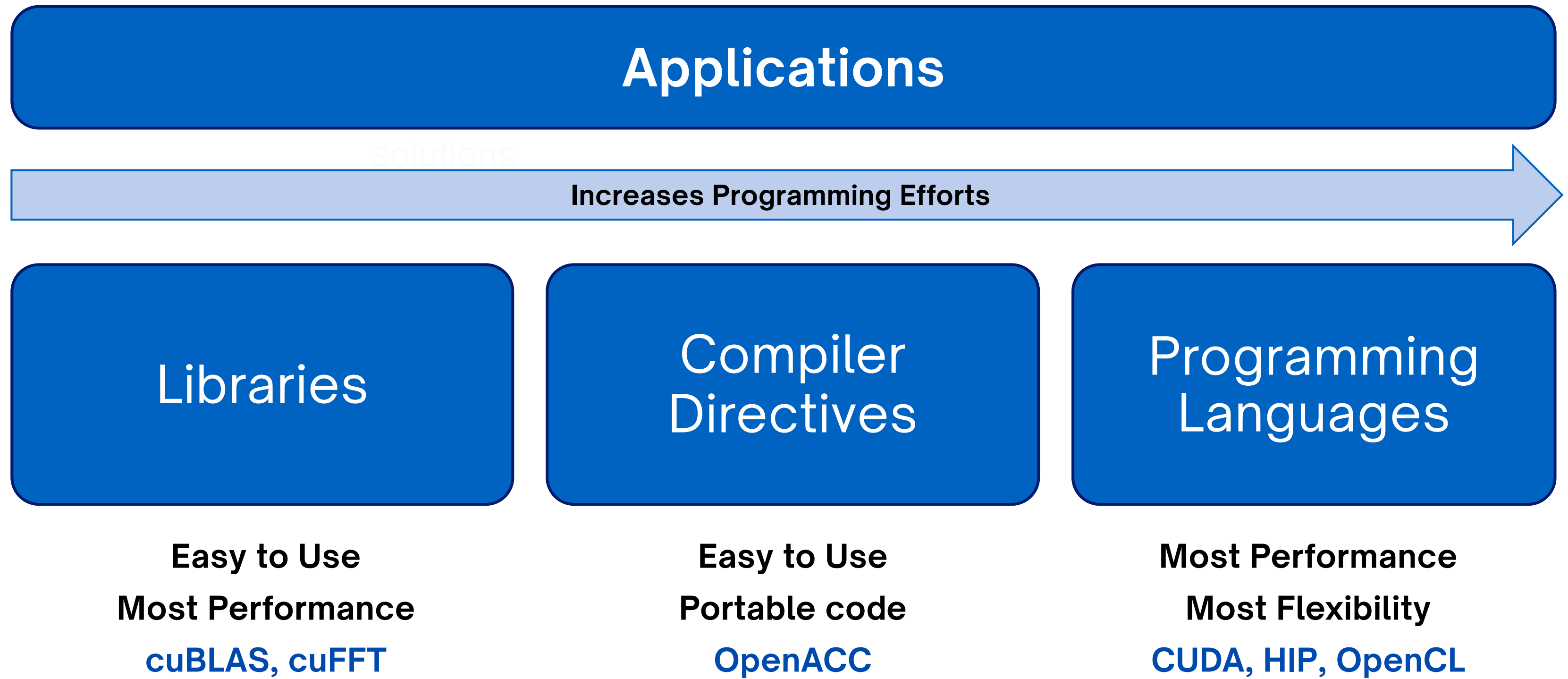
CloverLeaf

Comp Hydrodynamics
AWE
4X speedup
Single CPU/GPU code



FINE/Turbo

CFD
NUMECA
International
10X faster routines
2X faster app



OpenACC :

- Directive-based GPU offloading which requires minimal code change.
- Portable across vendors (compiler-dependent).
- Lower optimization control, compiler handles data movement.



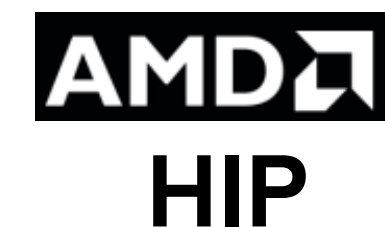
CUDA :

- Explicit GPU programming model (kernels, threads, memory)
- Highest performance on NVIDIA GPUs.
- Rich ecosystem: cuBLAS, cuFFT, cuDNN, Nsight
- Not portable



HIP :

- CUDA-like programming model with portability across AMD & NVIDIA GPUs
- Easy migration from CUDA using automatic hipify tools.
- Backed by the ROCm ecosystem with optimized math & ML libraries



C /C++

```
#pragma acc directive clauses  
<code>
```

FORTRAN

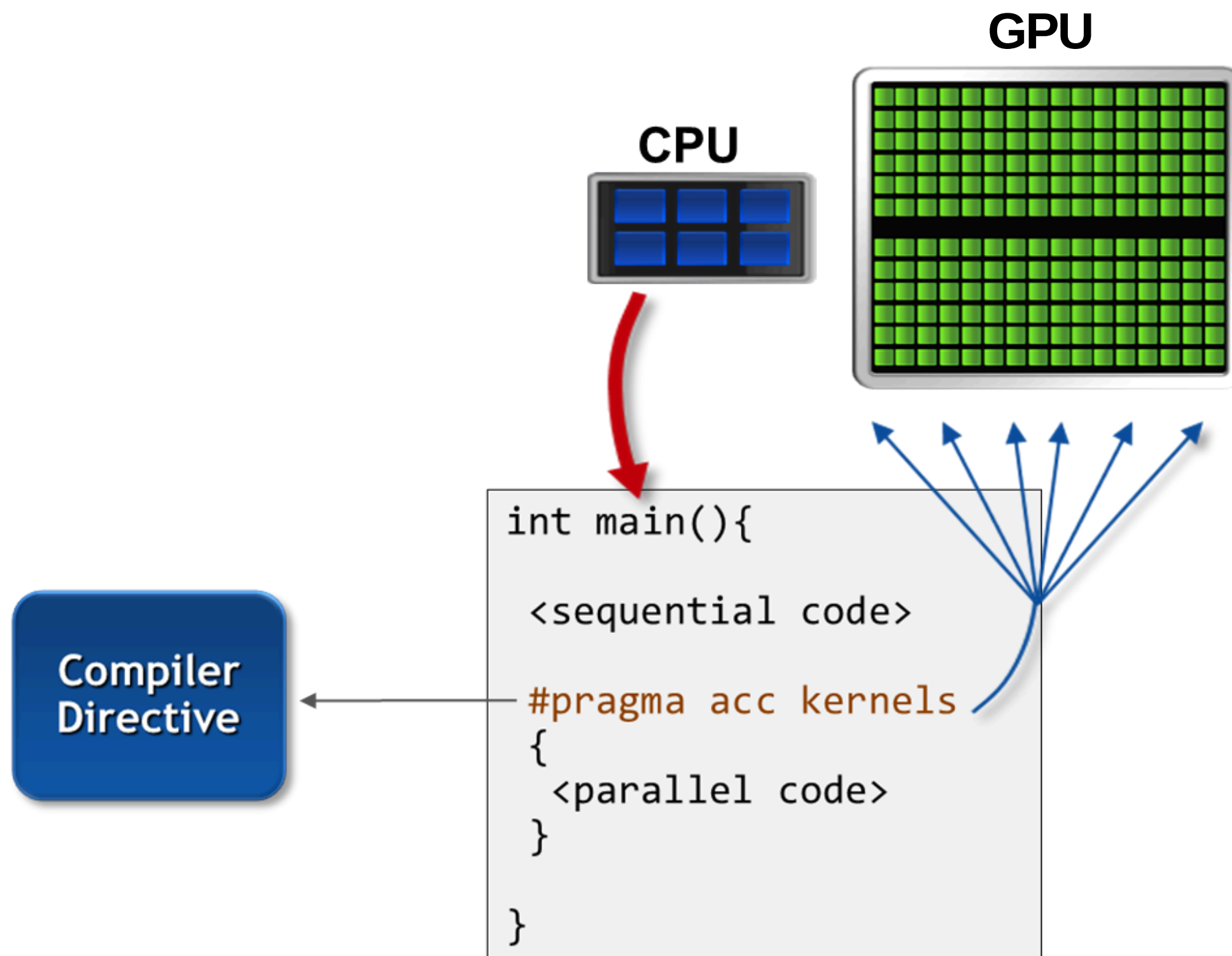
```
!$acc directive clauses  
<code>
```

- A pragma in C/C++ gives instructions to the compiler on how to compile the code.
- A **directive** in Fortran is a specially formatted comment that likewise instructions the compiler in its compilation of the code and can be freely ignored.
- “**acc**” informs the compiler that what will come is an OpenACC directive
- **Directives** are commands in OpenACC for altering our code.
- **Clauses** are specifiers or additions to directives.

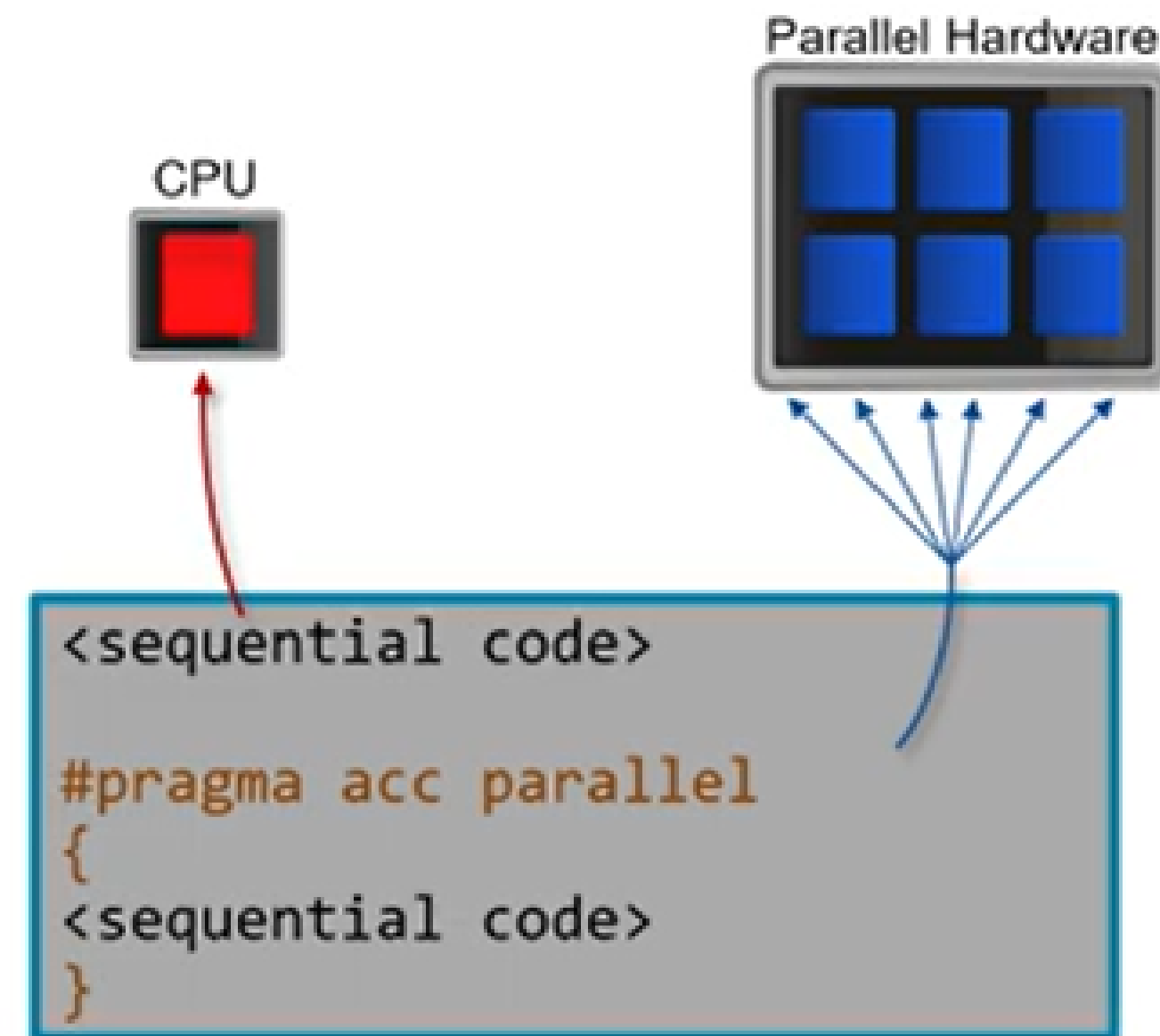
OpenACC Directive

The programmer will give hints to the compiler about which parts of the code to parallelize.

The compiler will then generate parallelism for the target parallel hardware



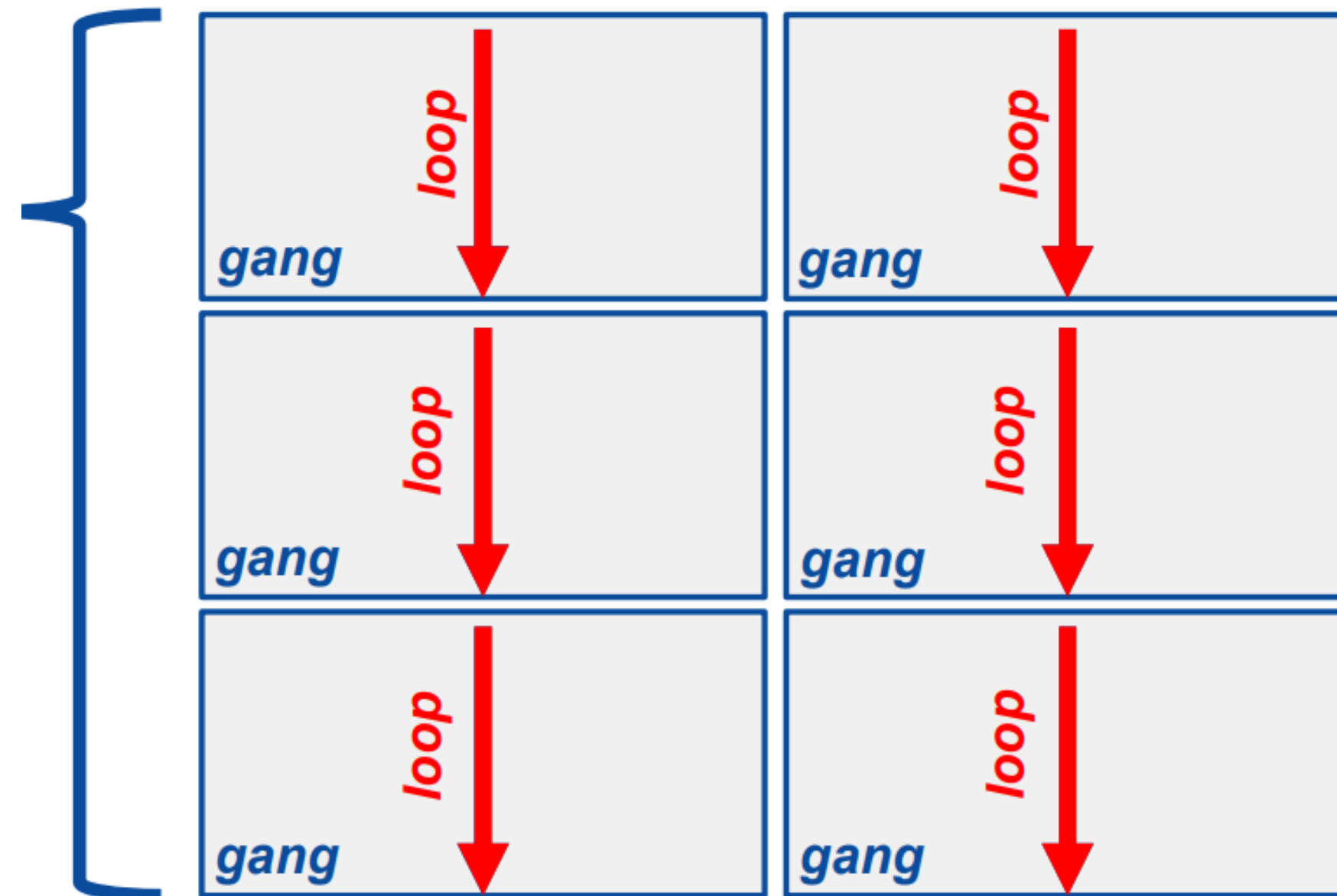
- The parallel directive instructs the compiler to create parallel gangs on the accelerator..
- Gangs are independent groups of worker threads on the accelerator.
- The code contained within a parallel directive is executed redundantly for all parallel gangs.



C++

```
#pragma acc parallel
{
    for (int i = 0; i < N; i++)
    {
        // Do Something
    }
}
```

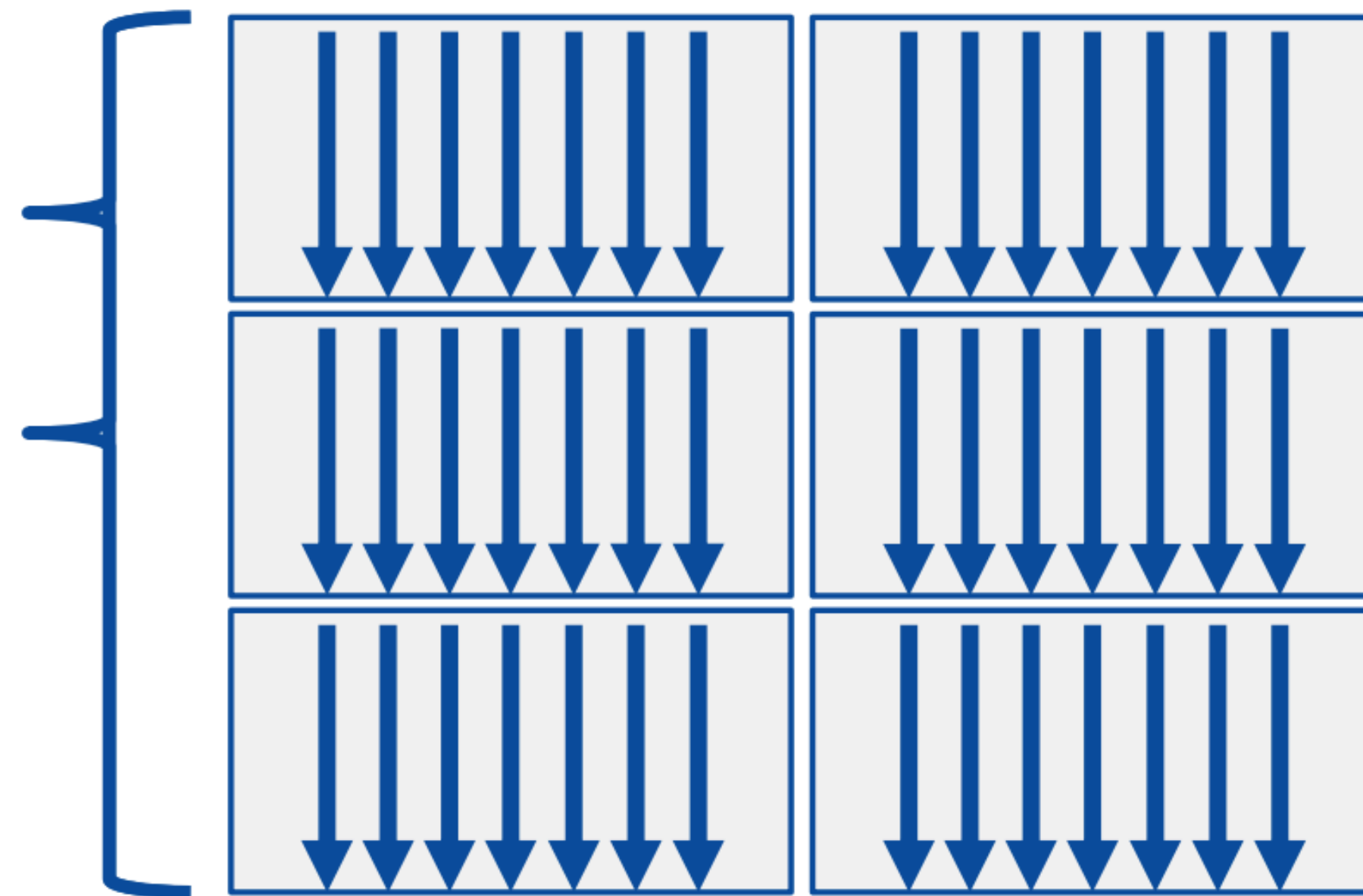
This means that each **gang** will execute the entire loop



C++

```
#pragma acc parallel
{
    #pragma acc loop
    for (int i = 0; i < N; i++)
    {
        // Do Something
    }
}
```

Distribute loop iterations
across gangs/workers



parallel

- The parallel directive identifies a region of code that will be parallelized across OpenACC gangs.
- Unlike the kernels directive, each loop needs to be explicitly decorated with parallel loop directives
- This is because the parallel directive relies on the programmer to identify the parallelism in the code rather than performing its own compiler analysis of the loops.
- ***parallel*** directive gives more control than kernels

C++

```
#pragma acc parallel loop
    for (i=0; i<N; i++)
    {
        y[i] = 0.0f;
        x[i] = (float)(i+1);
    }

#pragma acc parallel loop
    for (i=0; i<N; i++)
    {
        y[i] = 2.0f * x[i] + y[i];
    }
```

kernels

- The kernels directive identifies a region of code that may contain parallelism, but relies on the automatic parallelization capabilities of the compiler.
- Each separate loop nest inside the region will be converted into a separate parallel kernel.
- The compiler is given complete freedom to determine how best to map the parallelism available in these loops to the hardware.
- We don't want to think about thread levels, gangs, or vectors.

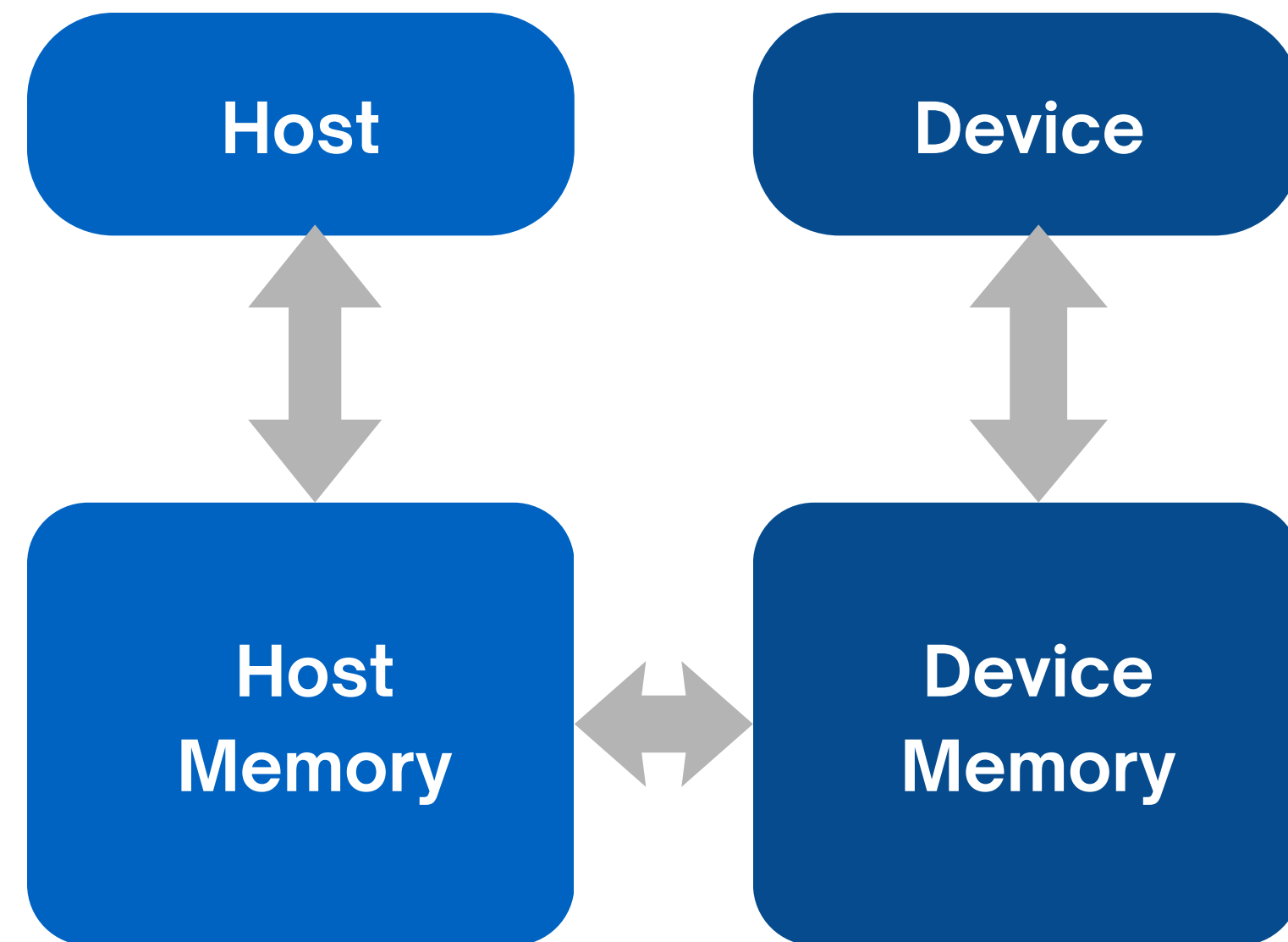
C++

```
#pragma acc kernels
{
    for (int i = 0; i < N; i++)
        A[i] = B[i] + 1;    // Kernel 1

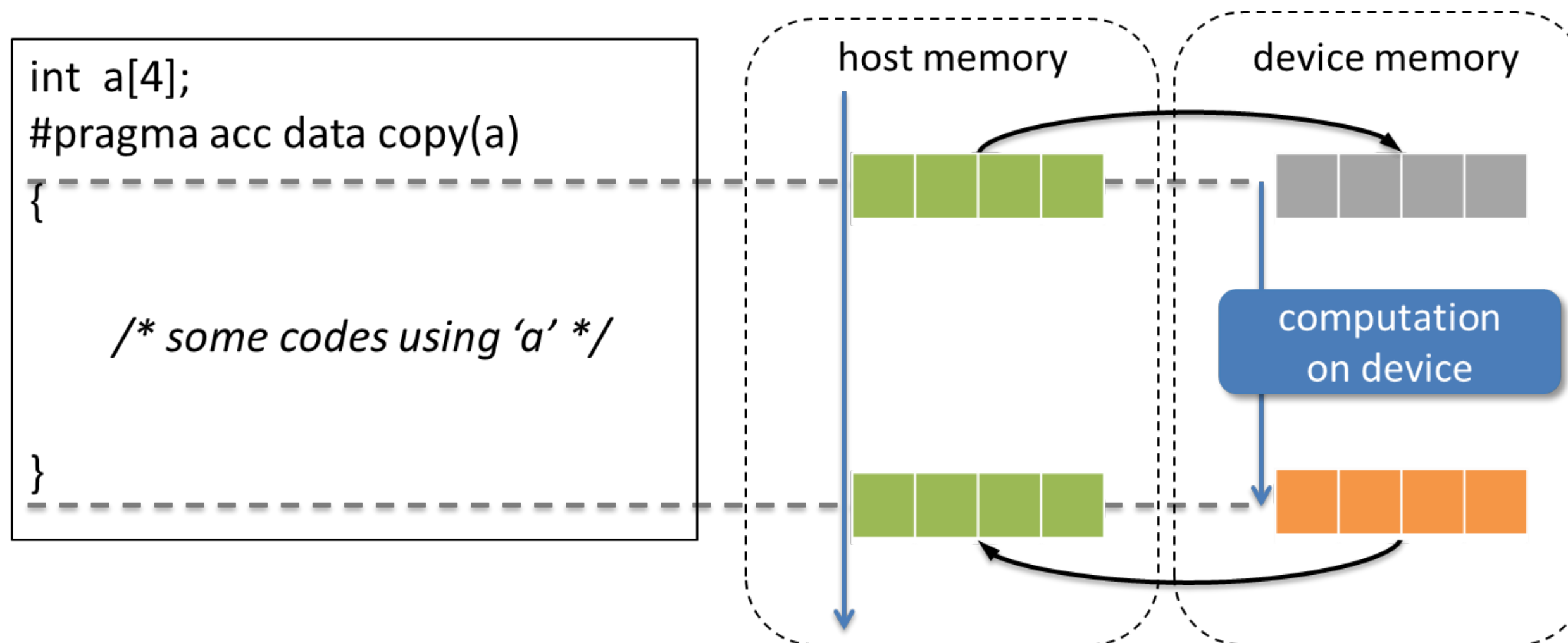
    for (int i = 0; i < N; i++)
        C[i] = A[i] * 2;    // Kernel 2
}
```

Data Management & Execution Model

- Many Parallel Accelerators (Devices) have a separate memory space from the Host.
- These separate memories contain completely different data.
- Data must be visible on the device when we run our parallel code.
- When the Host and Device don't share memory, data movement must occur.
- Transferring between these two memories can be a very time consuming process.



Data Management on Accelerator :



- If an array is specified in **"copy"** clause :

- Device Memory Allocation
- Data transfer from Host to Device
- Data transfer from Device to Host
- Device Memory Release

at the beginning of region

at the end of region

- The data directive defines a lifetime for data on the device.
- During the region data should be thought of as residing on the accelerator.
- Data clauses allow programmer to control the allocation and movement of data.

C /C++

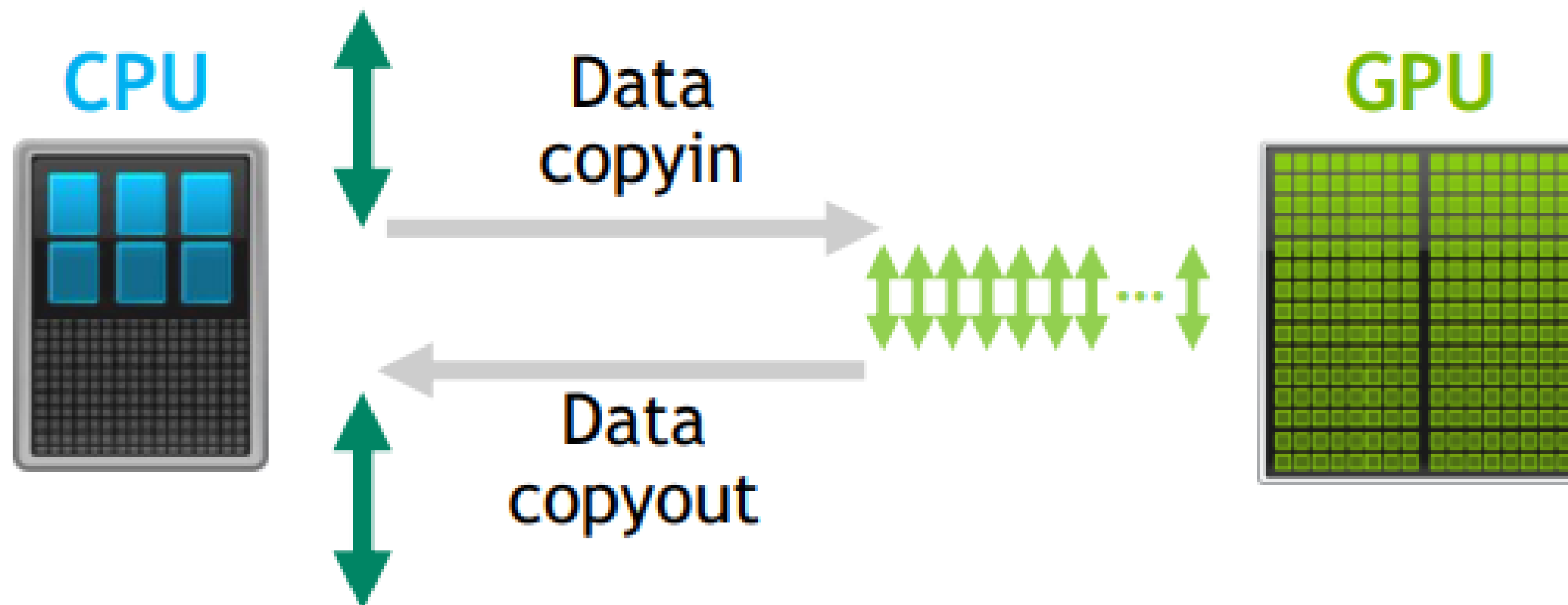
```
#pragma acc data clauses  
{  
    <Sequential and/or Parallel code>  
}
```

FORTRAN

```
!$acc data clauses  
    <Sequential and/or Parallel code>  
  
!$acc end data
```

Clause	Description
copy(list)	Allocates memory on device and copies data from Host to Device when entering region and copies data to the host when exiting region
copyin(list)	Allocates memory on device and copies data from Host to Device when entering region.
copyout(list)	Allocates memory on device and copies data to the Host when exiting region.
create(list)	Allocates memory on device but does not copy.

OpenACC Data Directive (Clauses)



Array Shaping

- Sometimes the compiler needs help understanding the shape of an array.
- The first number is the start index of an array.
- In C/C++, the second number is how much data is to be transferred.
- In Fortran, the second number is the ending index.

C

```
copy(array[starting_index:length])
```

FORTRAN

```
copy(array[starting_index:ending_index])
```

Loop Optimization

auto Clause

- The **auto** clause lets the compiler automatically decide how to execute a loop on the accelerator.
- We doesn't need to manually specify the gangs, workers, vectors.
- Compiler chooses the best configuration based on the loop size, data, and GPU architecture.
- Ensures maximum parallelism without worrying about low-level GPU details.

C /C++

```
#pragma acc parallel loop auto
for(int i = 0; i < N; i++) {
    a[i] = b[i] + c[i];
}
```

independent Clause

- Tells the compiler that all iterations of a loop are independent.
- Compiler can safely parallelize the loop without worrying about dependencies.
- **independent** clause guarantees that iterations don't depend on each other.
- Often used with auto or other execution clauses:

C /C++

```
#pragma acc kernels loop independent  
for(int i = 0; i < N; i++) {  
    a[i] = b[i] + c[i];  
}
```


seq Clause

- seq tells the compiler to execute the loop sequentially, not in parallel.
- It is useful when :
 - Loop iterations cannot run in parallel
 - You want to disable parallelization for specific loops inside a kernels region
- **seq** clause ensures iterations are executed one after another on the accelerator.
- Prevents the compiler from parallelizing loops with dependencies.
- **seq** is mainly used with kernels regions.

C /C++

```
#pragma acc kernels loop seq
for(int i = 1; i < N; i++) {
    a[i] = a[i-1] + b[i];
}
```

Dependent
Iteration

private Clause

- The **private** clause creates a separate copy of the variable for each gang/worker/vector (depending on the region).
- The variable is uninitialized inside the accelerator region.
- After the region ends, the value is not copied back to the host.
- Use private when :
 - Each thread needs its own temporary variable.
 - The variable should not be shared between iterations.
 - The variable does not need initial value.

C /C++

```
#pragma acc parallel loop private(temp)
for (int i = 0; i < N; i++) {
    temp = i * 2;
    a[i] = temp + 5;
}
```

// each iteration
has its own private
'temp'

firstprivate Clause

- **firstprivate** is similar to private, but initializes the private variable with the value from the host CPU before entering the accelerator region.
- Use firstprivate when :
 - Each thread needs its own copy.
 - You want each copy to start with the same initial value as the host variable.

C /C++

```
int x = 10;  
  
#pragma acc parallel loop firstprivate(x)  
for (int i = 0; i < N; i++) {  
    a[i] = x + i;  
}
```

Each thread gets
its own 'x',
initialized to 10